[Group 76] An Evidence-Grounded Information-Gathering Agent for Heterogeneous Data Lakes

Member 1	**Member 2**	**Member 3**
Student ID: [student ID]	Student ID: [student ID]	Student ID: [student ID]
[HKU email]	[HKU email]	[HKU email]

1 Project Objective

We aim to develop an end-to-end information-gathering agent that answers complex user requests requiring evidence from multiple heterogeneous data sources in a data lake. Given a natural-language request, the agent will decompose the task, discover relevant datasets or files, inspect schemas and contents, select suitable tools for structured queries and unstructured-text analysis, integrate evidence across sources, and verify the answer before returning a cited response. We will evaluate both dataset-grounded task performance (answer accuracy, numerical or structured-result accuracy, relevant-source retrieval precision, recall, and F1, and file-level evidence grounding) and execution-level behavior (task completion, valid and unnecessary tool calls, recovery from errors, latency, and token or external-data access cost). Baselines will isolate the contributions of planning, tool selection, multi-step execution, and evidence verification.

2 Motivation and Challenges

Complex information requests over data lakes are difficult because the relevant source is often unknown before execution. A single request may require finding several files, combining structured tables with unstructured documents, and using different operations for filtering, aggregation, and text extraction. Sources can have incompatible schemas, incomplete metadata, or conflicting evidence, while downloading and processing unnecessary data increases latency and cost. These conditions make a fixed-retrieval or one-shot answering pipeline unreliable.

The project therefore addresses five connected challenges: (i) decomposing an underspecified request into executable subtasks; (ii) discovering and ranking relevant datasets or files; (iii) choosing and sequencing tools for heterogeneous formats; (iv) aligning evidence and handling conflicts across sources; and (v) verifying the final answer while controlling retries and resource use. The system must also expose enough execution information to measure reliability and efficiency rather than reporting answer quality alone.

3 Potential Solutions

The proposed system contains five modules connected by explicit intermediate records:

1. **Planner.** Convert the user request into subtasks and dependencies, while recording the information required to answer each subtask.
2. **Source discovery and tool router.** Search the data-lake catalog, rank candidate sources using metadata and query relevance, and select operations appropriate to the source format.
3. **Deterministic executor.** Run bounded calls for discovery, download, inspection, structured querying, and text extraction. Each call records its input, output, latency, and error status.

4. **Evidence integrator and verifier.** Align results across subtasks, retain source references, check numerical and textual consistency, and trigger a bounded retry or an explicit insufficient-evidence response when verification fails.
5. **Trace and cost manager.** Enforce budgets for tool calls, tokens, retries, and external-data access, and export a complete trace for evaluation.

For controlled comparisons, we will implement a fixed top-k retrieval plus one-shot answering baseline, a planner without verification, and the full planner–executor–verifier system. The ablations will keep the task set and model configuration fixed so that changes in answer quality, evidence retrieval, and execution cost can be attributed to the corresponding module.

4 Experimental Dataset Description and Evaluation Plan

The primary benchmark will be LakeQA, using its public mini task set and task-level or on-demand access to the files required by each task. LakeQA represents data-lake exploration with heterogeneous CSV, JSON, text, PDF, and HTML resources and exposes operations such as `search`, `listdata`, `download`, `inspect`, and `query`. Each task supplies a natural-language question and an annotated answer; where available, its relevant-document annotation will be used for source-level grounding. We will freeze the task manifest, record file identifiers and access timestamps, and avoid claiming that the entire multi-terabyte lake is downloaded or processed.

We will report dataset-grounded metrics separately from execution-level metrics. For each task, we will compute exact-match or normalized answer accuracy, numerical or structured-result accuracy where applicable, and precision, recall, and F1 over retrieved relevant documents. Evidence grounding will be reported at the granularity supported by the annotations (primarily file-level). From execution traces and repeated runs under a fixed model and hardware configuration, we will compute task completion rate, valid and unnecessary tool-call rates, average tool calls, retry recovery rate, latency, token usage, and external-data access cost. Means and variability across tasks or repeated runs will be reported. Qualitative inspection of failed traces will characterize missed sources, tool errors, evidence conflicts, and unsupported answers.